

MetaControls

The magic of draft forms is given to you for programmatic use, piece by piece. The forms are built using MetaControls, which too are automatically generated, one for each table. Every MetaControl contains methods to create input controls for each column in the table. So, for example, if you want to create a page that contains the text-box for entering a blog post title from your example database, all you would need to do is:

```
// Create a textbox on the page
$this->txtTitle = new QTextBox($this);

//Create MetaControl for new Post
$mctPost = PostMetaControl::Create($this, null, @MetaControlCreat
eType::CreateOnRecordNotFound);
// Get input control from MetaControl's creator function
$this->txtTitle = $mctPost->txtTitle_Create();
```

And the textbox 'txtTitle' would contain all the limitations for the *title* column of *post* table in place; just render it! Now how can you get it done easier?

If the above code does not make any sense, you can either have a look at it in the second part of the Qcubed examples site, or wait for the next article in the series, when we cover how Qcubed helps you build Web pages real fast.

MetaControls are created depending on the data-type of each column. For example, if the data type is *varchar*, then the control would be a *textbox*; if it is *text*, the control would be a *textarea* element; if it is *Boolean*, the control would be a *checkbox*; and if it is a *foreign* key, it will be a drop-down select element. The MetaControls also contain information needed for validation, so that a form submission does not result in an error from the database. They are available for programmatic use, and you are probably never going to write validation logic for database columns again.

In addition to the goodies mentioned above, Qcubed also generates an AJAX control panel, which brings all the forms to one place, and can easily serve as the central control panel for all data you use in your Web app.

DataGrids

If you want to find all the posts that contain the word 'Google' in the title and the word 'California' in the body, perhaps the only way would be to write the appropriate query. Even with tools like phpMyAdmin, the process is not too simple—you either have to write queries or select tables and columns manually before you can query the database. Well, it might not be needed with Qcubed since it comes with controls called *QDataGrids*, which make searching in tables a lot easier than it would be otherwise. During the Codegen process, each table also receives its own *MetaDataGrid*, which can be used to present the table in the database as a table on the screen.

QDataGrids are limited neither to a single table nor to Codegen. You can create one of your own to present data

ID	Title	Post	Comment Body	Comment Date
1	Post Object 2	Post Object 2	This is a comment	2012-01-01
2	Post Object 2	Post Object 2	another comment	2012-01-01
3	Post Object 1	Post Object 1	comment on the first post	2012-01-01
4	Post Object 2	Post Object 2	Work comment on 2nd post	2012-01-01
5	Post Object 1	Post Object 1	second comment on 1st post	2012-01-01
6	Post Object 2	Post Object 2	third comment on 2nd post	2012-01-01

[Create a New Comment](#)

Figure 2: QDataGrids: Easy-to-use tables with search options

from multiple tables. You can select which columns you want to be sortable and/or searchable as well. However, there are a couple of drawbacks with QDataGrids:

- You cannot use the search or sort feature across columns that belong to two different tables. But you can display data from two tables in one QDataGrid using the templating feature they possess.
- You cannot create a QDataGrid using a hand-written query—you must use Qcubed's built-in database querying mechanism called QQuery. This is because QDataGrids cannot work on raw result sets in array format. They work only with objects.

How to run Codegen

Once you have the framework configured, running Codegen is straightforward. To configure the framework, you need to open the 'includes/configuration/configuration.inc.php.sample' file and change the values for `__DOCROOT__`, `__VIRTUAL__`, `__DIRECTORY__`, and `__SUBDIRECTORY__`. Also, you need to set the database parameters in `DB_CONNECTION_1`. "After making the changes, save the file as 'includes/configuration/configuration.inc.php'. Now, you need to open the base Qcubed installation address from the browser. That is to say, if your *DocumentRoot* directory is */var/www* and you installed Qcubed in */var/www/lfy* then you should open the address *http://localhost/lfy*. If you have not configured something right, the automated configuration checker will try to help you out.

You should see a 'Start' page showing the current configuration, details about the framework and important links to help you. It would also let you know of any other problems detected—for instance, if you entered wrong parameters for database configuration, that would be shown here as well. In addition, if there are file-system permission issues, that too is reported. Qcubed needs permissions to create and alter files in certain directories where it would put the Codegen files. The file-system permission issues are checked during the config-check as well! On the 'Start' page, you get the link to run the Codegen, so click on it. I recommend that you take a backup of the *index.php* file that comes along before modifying or replacing it with one of your own, as it contains important links you would need time and again.

Once you have configured the framework with one database, you can add more databases to your app defined as `DB_CONNECTION_2`, `DB_CONNECTION_3` and so on. Remember, however, that simply adding database connections would not run Codegen for all of them. You need to add Codegen parameters for each of them in the *codegen_settings.xml* file (in the same directory), in which various options and their effects have been